SmartCity360: Generation Prompt and Run Guide

For JIAT/OODP II/EX/01. Part 1 is the prompt: copy everything between BEGIN PROMPT and END PROMPT into your code generator. Part 2 is the guide to get the generated project running.

Part 1: Generation prompt

BEGIN PROMPT

1. Role and output rules

You are a senior Jakarta EE engineer. Generate a complete, compile-ready single-module Maven project called SmartCity360, a smart city management platform for a university object-oriented design patterns assignment.

Output rules:

- Output only project files. For each file print a line `FILE: <relative path>` followed by exactly one fenced code block holding the full file content. Follow the file tree in section 5, in order.
- No explanations, no UML, no report text, no commentary between files. README.md is the only documentation file.
- Every file must be complete: no TODO, no placeholders, no '...', no omitted imports, no pseudo-code.
- The project must build with `mvn clean package` on JDK 21 with zero manual edits, and all unit tests must pass.
- If you reach your output limit, stop at the end of a file, write `CONTINUE FROM: <next path>`, and resume when I reply 'continue'.

2. Stack and versions

- Java 21 (`maven.compiler.release` = 21). Payara 7 needs JDK 21 as a minimum.
- Jakarta EE 10 API `jakarta.platform:jakarta.jakartaee-api:10.0.0`, scope provided. Use only `jakarta.*` imports, never `javax.*`.

- JPA (EclipseLink supplied by Payara) on MySQL 8. Persistence unit `smartcity360PU`, JTA data source `jdbc/smartcity360` (I create the pool myself). The schema comes from sql/schema.sql, so schema generation is set to none.
- JAX-RS under `/api`, JSON-B, @Stateless EJB services and DAOs, DAO layer over EntityManager.
- Frontend: HTML5, CSS3, vanilla JS using fetch. No frameworks, no CDN links, no build tools.
- Tests: JUnit Jupiter 5.10.2. Plugins: maven-compiler-plugin 3.13.0, maven-surefire-plugin 3.2.5, maven-war-plugin 3.4.0.
- Base package `lk.jiat.smartcity360`. WAR finalName `smartcity360`, so the context root is /smartcity360.
- REST DTOs are plain classes with getters and setters (no records). Never return JPA entities from REST.

3. Engineering rules (avoid the course anti-patterns)

- No God classes. One responsibility per class. Methods of at most 25 lines.
- No magic numbers or strings: use enums and a constants class per feature.
- No swallowed exceptions: catch only to wrap in a custom exception (keep the cause) or to log with context and return an error. Never an empty catch.
- No copy-paste: extract shared logic. No tight coupling: depend on interfaces, use constructor injection in core, and no static singletons except the streetlight model registry.
- Model domain concepts with types (Severity, ZoneLevel, StaffRole, SensorType, LampState) instead of raw int or String.
- Organise by feature. No package, folder or class name uses a pattern name. Add a one-line Javadoc to the key interface and classes of each feature stating its role.

4. Pattern rules

Parts A to E each use exactly one pattern, and no other GoF pattern (no Factory, Builder, Strategy, Observer, Singleton and so on) appears in those feature packages. Implement them in the style of the lecture note: the abstract wrapper holds the wrapped object through its constructor; the composite holds a list of children; the handler holds `next` with a setter and getter; the flyweight pool hands out shared instances while each object keeps its own extrinsic data; expression classes combine into a tree and `interpret` it.

Part	Feature package	Pattern
A	energy	Decorator
B	zone	Composite
C	complaint	Chain of Responsibility
D	streetlight	Flyweight
E	policy	Interpreter

The supporting modules (sensor, waste, resident and staff, login) are plain DAO + service + resource with no patterns.

5. File tree (generate exactly this)

```
smartcity360/
├── pom.xml
├── README.md
├── .gitignore
├── sql/
│   ├── schema.sql
│   └── seed-data.sql
└── src/
    ├── main/
    │   ├── java/lk/jiat/smartcity360/
    │   │   ├── config/        RestApplication, StartupSeeder,
ApiExceptionMapper, ErrorResponse
    │   │   ├── security/      AuthResource, SessionRegistry, AuthFilter,
RequiresRole,
    │   │   │                  PasswordHasher, LoginRequest, LoginResponse
    │   │   ├── persistence/
    │   │   │   ├── entity/    AppUserEntity, ZoneEntity, EnergyMeterEntity,
MeterReadingEntity,
    │   │   │   │             StaffMemberEntity, ResidentEntity,
ComplaintEntity,
    │   │   │   │             ComplaintTraceEntity, StreetlightModelEntity,
StreetlightEntity,
    │   │   │   │             SensorEntity, SensorReadingEntity,
WasteBinEntity,
    │   │   │   │             PolicyEntity, PolicyLogEntity
    │   │   │   ├── dao/       GenericDao + one DAO per entity (ZoneDao,
```

```
ComplaintDao, ...)
    |   |   |   └── exception/ DataAccessException
    |   |   ├── energy/        plain Java: MeterStream, RawMeterStream,
StreamLayer,
    |   |   |                  EncryptionLayer, AuthenticationLayer,
CompressionLayer, LayerType,
    |   |   |                  StreamLayerAssembler, StreamSecurityConfig,
    |   |   |                  StreamAuthenticationException, EnergyConstants
    |   |   |                  web: EnergyResource, EnergyService + DTO
classes
    |   |   ├── zone/          plain Java: ZoneNode, Zone, Building,
ZoneLevel
    |   |   |                  web: ZoneResource, ZoneService + DTO classes
    |   |   ├── complaint/     plain Java: CitizenComplaint,
ComplaintHandler, LocalStaffHandler,
    |   |   |                  SupervisorHandler, CityManagerHandler,
EscalationChainAssembler,
    |   |   |                  EscalationStep, Severity, StaffRole,
ComplaintStatus
    |   |   |                  web: ComplaintResource, ComplaintService,
EscalationSettings + DTOs
    |   |   ├── streetlight/   plain Java: StreetlightModel,
StreetlightModelRegistry,
    |   |   |                  SmartStreetlight, LampState
    |   |   |                  web: StreetlightResource, StreetlightService +
DTO classes
    |   |   ├── policy/        plain Java: PolicyExpression, ValueExpression,
PolicyContext,
    |   |   |                  PolicyEngine, PolicyParser, PolicyTokenizer,
PolicySyntaxException,
    |   |   |                  ComparisonOperator, NumberLiteral,
VariableReference, TimeWindow,
    |   |   |                  ComparisonExpression, AndExpression,
OrExpression, NotExpression
    |   |   |                  web: PolicyResource, PolicyService + DTO
classes
    |   |   ├── sensor/        SensorResource, SensorService + DTO classes
    |   |   ├── waste/         WasteResource, WasteService + DTO classes
    |   |   └── resident/      ResidentResource, ResidentService + DTO
classes
    |   ├── resources/META-INF/persistence.xml
    |   └── webapp/
    |       ├── login.html
```

```
    |       ├── index.html
    |       ├── css/style.css
    |       ├── js/   api.js, auth.js, energy.js, zones.js, complaints.js,
streetlights.js,
    |       |         policies.js, sensors.js, waste.js, residents.js
    |       └── WEB-INF/web.xml
    └── test/java/lk/jiat/smartcity360/
        ├── energy/        EnergyStreamTest
        ├── zone/          ZoneTest
        ├── complaint/     ComplaintChainTest
        ├── streetlight/   StreetlightRegistryTest
        └── policy/        PolicyParserTest
```

One DTO class per file inside each feature package.

6. Maven setup

- One pom.xml, packaging war, no parent POM and no modules. UTF-8, release 21. Dependencies: jakartaee-api 10.0.0 (provided) and JUnit Jupiter 5.10.2 (test). Plugins: maven-compiler-plugin, maven-surefire-plugin and maven-war-plugin (failOnMissingWebXml false, finalName smartcity360).
- src/main/resources/META-INF/persistence.xml lists every entity class explicitly, uses JTA data source `jdbc/smartcity360`, and sets `jakarta.persistence.schema-generation.database.action` to none. Entity classes end in Entity and DAOs end in Dao, so they never clash with the plain-Java domain classes of the same name (Zone, StreetlightModel and so on).
- Layering inside the one module: the plain-Java classes of the energy, zone, complaint, streetlight and policy packages (everything except the Resource, Service and DTO classes) have no jakarta.* imports and no dependency on the persistence package. Resources, services, DTOs and DAOs depend on them, never the reverse. This keeps the unit tests runnable without a server or database.
- web.xml only sets the welcome file login.html.

7. Database (MySQL 8, database smartcity360)

Create sql/schema.sql with exactly these tables (InnoDB, utf8mb4, BIGINT AUTO_INCREMENT primary keys named id unless stated, foreign keys declared):

```
app_user(username UNIQUE, password_hash, role ADMIN|STAFF|RESIDENT,
full_name)
zone(name, level CITY|DISTRICT|STREET|BUILDING, parent_id NULL FK zone,
powered BOOLEAN DEFAULT 1, base_usage_kwh DECIMAL(12,2) DEFAULT 0)
energy_meter(meter_code UNIQUE, zone_id FK, layers VARCHAR(100),
installed_at)
meter_reading(meter_id FK, reading_kwh DECIMAL(12,3), raw_size INT,
secured_size INT, layers_applied VARCHAR(100), recorded_at)
staff_member(full_name, role LOCAL_STAFF|SUPERVISOR|CITY_MANAGER, email)
resident(full_name, email UNIQUE, address, zone_id FK)
complaint(resident_id FK, title, description TEXT, severity
LOW|MEDIUM|HIGH|CRITICAL, status OPEN|RESOLVED|UNRESOLVED,
resolved_by_staff_id NULL FK, created_at, resolved_at NULL)
complaint_trace(complaint_id FK, step_no INT, handler_role, handler_name,
action HANDLED|ESCALATED|UNRESOLVED, note, logged_at)
streetlight_model(model_code VARCHAR(40) PRIMARY KEY, lamp_type, wattage INT,
max_lumens INT, dim_profile, manufacturer)
streetlight(model_code FK, zone_id FK, latitude DECIMAL(9,6), longitude
DECIMAL(9,6), state ON|OFF|DIMMED, brightness_percent INT)
sensor(sensor_code UNIQUE, sensor_type NOISE|PARKING|TEMPERATURE, zone_id FK,
active BOOLEAN)
sensor_reading(sensor_id FK, value DOUBLE, recorded_at)
waste_bin(bin_code UNIQUE, zone_id FK, location_desc, capacity_litres INT,
fill_percent INT, last_collected_at NULL)
policy(name UNIQUE, rule_text VARCHAR(500), action_text VARCHAR(200), enabled
BOOLEAN, updated_at)
policy_log(policy_id FK, triggered BOOLEAN, context_snapshot VARCHAR(500),
evaluated_at)
```

Create sql/seed-data.sql: one city with 2 districts, 2 streets per district and 2 buildings per street (8 buildings, each with a base_usage_kwh and one energy meter with layers `ENCRYPTION,AUTHENTICATION,COMPRESSION`); 3 staff (one per role); 3 residents; 5 streetlight models and at least 40 streetlights spread over the streets; 6 sensors (2 per type) each with 3 readings (include noise values above and below 70); 6 waste bins with mixed fill levels; 3 policies:

- noise_after_hours: `noise_level > 70 AND time_between(22:00, 06:00)`, action ALERT noise limit exceeded
- parking_full: `parking_occupancy >= 95`, action ALERT parking zone full
- heat_warning: `temperature > 35 OR (temperature > 32 AND noise_level > 60)`, action ALERT heat advisory

The app_user rows are NOT in seed-data.sql: StartupSeeder (a @Singleton @Startup EJB) inserts them when app_user is empty, hashing with PasswordHasher (PBKDF2WithHmacSHA256, random salt). Default users: admin / Admin@123 (ADMIN), staff / Staff@123 (STAFF), resident / Resident@123 (RESIDENT).

8. Feature specifications

Part A: energy (Decorator)

- `MeterStream` interface: `String getDescription()`, `byte[] transmit(String reading)`, `String receive(byte[] packet)`.
- `RawMeterStream` is the base: UTF-8 bytes in and out, description 'Raw meter stream'.
- `StreamLayer` is an abstract class implementing MeterStream, holding a protected MeterStream `inner` from its constructor.
- `EncryptionLayer`: AES/GCM with a random 12-byte IV prepended; key from `StreamSecurityConfig` (system property `smartcity360.stream.key` with a documented dev default held in a constant).
- `AuthenticationLayer`: appends an HMAC-SHA256 tag over the payload; `receive` verifies it and throws `StreamAuthenticationException` on mismatch.
- `CompressionLayer`: GZIP.
- Each layer's `transmit` post-processes `inner.transmit(...)`; `receive` reverses its step and then calls `inner.receive(...)`. Each description appends ' + <Layer name>' to the inner description.
- `LayerType` enum (ENCRYPTION, AUTHENTICATION, COMPRESSION). `StreamLayerAssembler.assemble(MeterStream base, List<LayerType> layers)` wraps in list order (plain class, not a named pattern).
- Web: each meter stores its layer list. POST reading runs the value through the stack, verifies the round trip, saves raw_size, secured_size and layers_applied. A preview endpoint shows sizes for any layer combination without saving.

Part B: zone (Composite)

- `ZoneNode` interface: `String getName()`, `ZoneLevel getLevel()`, `double getEnergyUsageKwh()`, `boolean isPowered()`, `void shutdownPower()`, `void restorePower()`, `String reportUsage()` (indented multi-line text showing the subtree).

- `Building` (leaf): holds its own usage and powered flag. A powered-off building reports 0 usage.
- `Zone` (composite, levels CITY, DISTRICT, STREET): holds a list of ZoneNode, with `add`, `remove`, `getChildren`. Usage, shutdown, restore and report recurse over children; `isPowered()` is true only if at least one child is powered.
- Web: ZoneService loads the zone table (self-referencing parent_id), builds the tree recursively, applies the command to the chosen node, then persists the powered flag of every affected row. The tree endpoint returns nested JSON with usage and power state per node.

Part C: complaint (Chain of Responsibility)

- `Severity` enum with a numeric rank (LOW 1 ... CRITICAL 4) defined as enum fields. `StaffRole` (LOCAL_STAFF, SUPERVISOR, CITY_MANAGER). `ComplaintStatus` (OPEN, RESOLVED, UNRESOLVED).
- `CitizenComplaint`: id, title, description, severity, status, list of `EscalationStep` (step number, role, handler name, action, note, time).
- `ComplaintHandler` abstract class with private `next`, `setNext`, `getNext`, abstract `handle(CitizenComplaint)`, and one protected helper used by all subclasses to pass the complaint on (marks UNRESOLVED and records the step if there is no next handler).
- `LocalStaffHandler` (max LOW), `SupervisorHandler` (max HIGH), `CityManagerHandler` (max CRITICAL). Each handles when severity rank is at most its maximum, otherwise records an ESCALATED step and passes on. Every hop adds an EscalationStep, so the trace is complete.
- `EscalationChainAssembler.assemble(List<StaffRole> order)` links handlers in the given order and returns the head. This is how the escalation path changes at runtime.
- Web: ComplaintService builds the chain from `EscalationSettings` (an @ApplicationScoped bean holding the current order, default LOCAL_STAFF, SUPERVISOR, CITY_MANAGER), runs the complaint, then saves the complaint and its trace rows. The staff member recorded as handler is the first staff_member with that role.

Part D: streetlight (Flyweight)

- `StreetlightModel`: immutable intrinsic state (modelCode, lampType, wattage, maxLumens, dimProfile, manufacturer).

- `StreetlightModelRegistry`: static ConcurrentHashMap keyed by modelCode; `getModel(...)` uses computeIfAbsent so each model exists once; also exposes `size()` and a request counter.
- `SmartStreetlight`: extrinsic state only (id, latitude, longitude, zoneId, LampState, brightnessPercent) plus a reference to its shared model; `turnOn`, `turnOff`, `dim(int percent)` with range validation against constants, and `describe()`.
- Web: StreetlightService loads lights joined to models and builds SmartStreetlight objects through the registry. `GET /streetlights/stats` returns total lights, unique models held by the registry, and an estimated memory comparison with and without sharing (per-model and per-light byte estimates held as named constants). `POST /streetlights/bulk?count=N` inserts N generated lights using JDBC-style batch flushing in the DAO, with a maximum held in a constant (20000).

Part E: policy (Interpreter)

- Two expression interfaces: `PolicyExpression { boolean interpret(PolicyContext ctx); }` and `ValueExpression { double interpret(PolicyContext ctx); }`.
- Terminals: `NumberLiteral`, `VariableReference` (looks up the variable in the context, throws a clear exception if missing), `TimeWindow` (true when the context time is inside the window, including overnight windows such as 22:00 to 06:00).
- Non-terminals: `ComparisonExpression` (two ValueExpressions plus `ComparisonOperator` GT, LT, GTE, LTE, EQ, NEQ), `AndExpression`, `OrExpression`, `NotExpression`.
- `PolicyContext`: Map of variable name to double plus a LocalTime.
- Grammar for `PolicyParser` (tokenizer plus recursive descent, `PolicySyntaxException` reports the position):

```
rule       := orExpr
orExpr     := andExpr { 'OR' andExpr }
andExpr    := unary { 'AND' unary }
unary      := 'NOT' unary | primary
primary    := '(' orExpr ')' | timeBetween | comparison
comparison := value op value
value      := NUMBER | IDENT
op         := '>' | '<' | '>=' | '<=' | '==' | '!='
timeBetween:= 'time_between' '(' TIME ',' TIME ')'
```

- Keywords are case-insensitive, TIME is HH:mm, AND binds tighter than OR.

- `PolicyEngine.evaluate(String ruleText, PolicyContext ctx)` parses and interprets.
- Web: policies are stored as text, so editing a rule changes behaviour immediately with no redeploy. PolicyService builds the context from the latest reading of each sensor type (variables `noise_level`, `parking_occupancy`, `temperature`) and the current time. Evaluate-all runs every enabled policy and writes a policy_log row with a context snapshot. Validate checks syntax only and returns the error position.

9. Supporting modules and login

- Sensor: sensors CRUD, record a reading, latest reading per sensor and per type. Waste: bins CRUD, update fill percent, list bins due for collection (fill at or above a constant threshold of 80), collect (resets fill to 0 and sets last_collected_at). Resident and staff: register and list residents, list staff.
- Login: POST /api/auth/login checks the password with PasswordHasher and returns a random UUID token held in an @ApplicationScoped `SessionRegistry` (in-memory, expires after a constant idle timeout). `AuthFilter` (ContainerRequestFilter, @Provider) requires `Authorization: Bearer <token>` on every /api path except login and reads the `@RequiresRole` annotation on resource methods. ADMIN can do everything. STAFF can do everything except methods marked ADMIN. RESIDENT gets GET endpoints plus POST /complaints.
- `ApiExceptionMapper` turns exceptions into JSON `{error, message}` with proper status codes (400 validation and syntax errors, 401, 403, 404, 500).

10. REST API (all under /api, JSON)

Area	Endpoints
Auth	POST /auth/login, POST /auth/logout, GET /auth/me
Energy	GET /energy/meters, POST /energy/meters (ADMIN), PUT /energy/meters/{id}/layers (ADMIN), POST /energy/meters/{id}/readings, GET /energy/meters/{id}/readings, POST /energy/preview

Area	Endpoints
Zones	GET /zones/tree, POST /zones (ADMIN), POST /zones/{id}/shutdown (ADMIN), POST /zones/{id}/restore (ADMIN), GET /zones/{id}/report
Complaints	POST /complaints, GET /complaints, GET /complaints/{id} (with trace), GET /complaints/chain, PUT /complaints/chain (ADMIN)
Streetlights	GET /streetlights (zoneId, page, size), GET /streetlights/stats, POST /streetlights (ADMIN), POST /streetlights/bulk (ADMIN), PUT /streetlights/{id}/state
Policies	GET /policies, POST /policies (ADMIN), PUT /policies/{id} (ADMIN), DELETE /policies/{id} (ADMIN), POST /policies/validate, POST /policies/evaluate-all, GET /policies/logs
Sensors	GET /sensors, POST /sensors (ADMIN), POST /sensors/{id}/readings, GET /sensors/latest
Waste	GET /waste/bins, POST /waste/bins (ADMIN), PUT /waste/bins/{id}/fill, GET /waste/bins/due, POST /waste/bins/{id}/collect
People	GET /residents, POST /residents, GET /staff

11. Frontend

- login.html posts credentials, stores the token in sessionStorage, and redirects to index.html. `api.js` wraps fetch, adds the Bearer header, and redirects to login on 401. `auth.js` hides ADMIN-only controls for other roles and shows a logout button.
- index.html is a single page with tabs: Energy, Zones, Complaints, Streetlights, Policies, Waste, Sensors, Residents. One JS file per tab. Clean, responsive CSS3 (grid or flexbox, readable tables, status badges), no external assets.
- Energy: layer checkboxes plus a reading input; a Preview button shows raw size versus secured size and the layer description; Send saves and shows the round-trip result.
- Zones: collapsible tree showing usage and power state per node; Shutdown, Restore and Report buttons work at every level (city, district, street, building).

- Complaints: submit form; list; click a complaint to see its escalation trace as a timeline; ADMIN can reorder the chain.
- Streetlights: paged table, stats panel (lights versus shared models and estimated memory saved), bulk-generate button.
- Policies: list, editor, Validate button (shows error position), Evaluate All button, log table.

12. Unit tests

- EnergyStreamTest: round trip for every layer combination and order; tampered packet fails authentication; compression shrinks repetitive input.
- ZoneTest: usage totals at every level; shutdown at a district zeroes its subtree only; restore works.
- ComplaintChainTest: each severity lands on the right handler; trace length is correct; a chain without a capable handler ends UNRESOLVED; reordered chain behaves differently.
- StreetlightRegistryTest: the same model code returns the same instance (assertSame); size counts unique models only.
- PolicyParserTest: precedence of AND over OR; NOT; parentheses; overnight time window; case-insensitive keywords; syntax errors report positions.

13. README.md

Write a README with: project summary, project structure overview, prerequisites (JDK 21, Maven 3.9, MySQL 8, Payara 7), database setup, Payara pool setup with asadmin commands, build and deploy, URLs, default logins, how to run the tests, and a troubleshooting list. It must match Part 2 of my guide: JDBC pool name SmartCity360Pool, resource jdbc/smartcity360, database smartcity360, MySQL user smartcity360 with password smartcity360123.

END PROMPT

Part 2: Run guide

Prerequisites

- JDK 21 or newer. Payara 7 does not run on JDK 17, so check with `java -version` and set JAVA_HOME if needed.

- Maven 3.9 or newer, MySQL 8, Payara Server 7, IntelliJ IDEA, and the MySQL JDBC driver jar (mysql-connector-j 8.x) from the MySQL website.

1. Create the database

In MySQL Workbench or the mysql client, as root:

```
CREATE DATABASE IF NOT EXISTS smartcity360 CHARACTER SET utf8mb4;
CREATE USER IF NOT EXISTS 'smartcity360'@'localhost' IDENTIFIED BY
'smartcity360123';
GRANT ALL PRIVILEGES ON smartcity360.* TO 'smartcity360'@'localhost';
FLUSH PRIVILEGES;
USE smartcity360;
SOURCE sql/schema.sql;
SOURCE sql/seed-data.sql;
```

If you use Workbench, open and run sql/schema.sql and then sql/seed-data.sql instead of the SOURCE lines.

2. Add the MySQL driver to Payara

Copy the connector jar into the domain lib folder, then restart:

```
copy mysql-connector-j-8.x.x.jar <payara>/glassfish/domains/domain1/lib/
<payara>/bin/asadmin restart-domain
```

3. Create the connection pool and JNDI resource

Run from `<payara>/bin` (use asadmin.bat on Windows; the commands are the same):

```
asadmin start-domain
asadmin create-jdbc-connection-pool --datasourceclassname
com.mysql.cj.jdbc.MysqlDataSource --restype javax.sql.DataSource --property
'serverName=localhost:portNumber=3306:databaseName=smartcity360:user=smartcit
y360:password=smartcity360123:useSSL=false:allowPublicKeyRetrieval=true:serve
rTimezone=UTC' SmartCity360Pool
asadmin create-jdbc-resource --connectionpoolid SmartCity360Pool
jdbc/smartcity360
asadmin ping-connection-pool SmartCity360Pool
```

The ping must print 'Command ping-connection-pool executed successfully'. (`javax.sql.DataSource` in --restype is the JDBC interface name and is correct even on Jakarta EE.) You can do the same in the admin console at http://localhost:4848 under Resources > JDBC.

4. Build

Open the smartcity360/pom.xml in IntelliJ (File > Open > select pom.xml > Open as Project), set the Project SDK to JDK 21, let Maven import, then run:

```
mvn clean package
```

This runs the unit tests and produces target/smartcity360.war.

5. Deploy

```
asadmin deploy --force target/smartcity360.war
```

IntelliJ Ultimate users can instead add a Payara Server run configuration (Run > Edit Configurations > + > Payara Server > Local), pick the smartcity360:war exploded artifact, and press Run. IntelliJ Community has no application-server run configuration, so use the asadmin command above (or the Payara admin console under Applications > Deploy).

6. Use the app

- Open http://localhost:8080/smartcity360/login.html
- Logins: admin / Admin@123, staff / Staff@123, resident / Resident@123
- Quick API check after logging in: GET http://localhost:8080/smartcity360/api/zones/tree with the Bearer token (Postman works well).

Troubleshooting

- UnsupportedClassVersionError or Payara fails to start: the runtime JDK is below 21. Point JAVA_HOME at JDK 21 and restart the domain.
- Lookup of jdbc/smartcity360 fails or persistence unit not found: the pool or resource is missing, or the pool ping fails. Re-run step 3 and check the driver jar is in domain1/lib and the domain was restarted.

- ClassNotFoundException for com.mysql.cj.jdbc.MysqlDataSource: the driver jar is not in domain1/lib.
- Any error mentioning javax.* packages: a generated file imported javax instead of jakarta. Replace the import and rebuild.
- Table not found: schema.sql was not run against smartcity360.
- 401 Unauthorized right after a redeploy: sessions are in memory and reset on every deploy, so log in again.
- Port 8080 or 4848 already in use: stop the other process or change the Payara listener port in the admin console.
- Maven cannot compile on JDK 17: set the Maven runtime JDK to 21 (IntelliJ: Settings > Build Tools > Maven > Runner and Importing > JRE).

Viva tip

For each part, open the feature package and walk the lecturer through three things: the interface or abstract class that defines the common contract, the concrete classes that implement or wrap it, and the line in the service that assembles them at runtime.